

F a c a d e

Facade Pattern

One simple door to a subsystem

DEFINITION

Provide a unified, simplified interface to a set of interfaces in a subsystem.

Facade puts a single front-desk object over a tangle of subsystem classes: clients ask the facade, not the dozen parts behind it. It knows which subsystem class handles each request and delegates — a unified, higher-level interface that makes the subsystem easier to use, without forbidding direct access for advanced clients.

WHY IT MATTERS

A subsystem with many fine-grained classes forces every client to learn the wiring and couples them to its internals. A facade gives one coarse API for the common cases, so clients depend on a stable front door and the internals can change freely behind it. It adds no behavior — if you are transforming data or enforcing rules, that is no longer a facade, and a facade that grows logic becomes a god object.

— How it works

Facade Knows which subsystem class handles a request and delegates.

Subsystem classes Do the real work; they have no knowledge of the facade.

Client Talks to the facade instead of the subsystem directly.

HOW TO APPLY

1. Identify the common tasks clients perform against the subsystem.
2. Add a Facade with one coarse method per task that orchestrates the parts.
3. Delegate to subsystem classes — don't add logic; keep it a thin front.
4. Leave direct access open for advanced clients (a transparent facade).

LITMUS TEST

Are clients wiring up many subsystem classes for a common task? If most callers want one simple default view, put a facade in front.

— Smell → fix

Every caller wires up the boot sequence by hand:

```
cpu.freeze()
mem.load(0, disk.read(0, 512)) // caller knows it all
cpu.jump(0)
cpu.exec() // every client repeats this dance
```

↓ ONE DOOR OVER THE SUBSYSTEM

```
class Computer { // the Facade
start() {
this.cpu.freeze()
this.mem.load(0, this.disk.read(0, 512))
this.cpu.jump(0); this.cpu.exec()
}
}
new Computer().start() // one simple call
```

Computer.start() now orchestrates cpu / mem / disk behind one call; clients learn one method, not the whole boot sequence. The internals can change freely — just don't let the facade absorb business logic, or it becomes a god object.

USE WHEN

- ✓ Tame a complex library
- ✓ Provide a coarse API for common tasks

AVOID WHEN

- ▲ Hiding so much it becomes a god object

— Trade-offs & pitfalls

PROS

- + Reduces coupling to the subsystem
- + Easier onboarding

CONS

- Can become a bottleneck / god class
- Hides power-user features

COMMON PITFALLS

- ▲ It can become a god object: a facade that accrues logic turns into a bloated do-everything class.
- ▲ It adds no behavior — if you are transforming data or rules, that is not a facade anymore.
- ▲ Coupling moves, not gone: the facade now depends on every subsystem it fronts; prefer a facade per layer over one mega-facade.

WHERE YOU SEE IT

- ▶ A compiler's compile() over lexer / parser / codegen (GoF's own); Node fs / fetch over syscalls and sockets.
- ▶ ORM repositories whose save() hides SQL, the connection pool, and transactions.
- ▶ API gateways, BFFs, and application-service layers — facades at architectural scale.

SEE ALSO

Adapter

makes one class fit an existing interface; Facade invents a new simpler one over many

Mediator

its behavioral cousin — but Mediator coordinates peers two-way; Facade is one-way

Proxy

stands in for one object; a Facade fronts a whole subsystem

Singleton

one facade is usually enough, so it is often kept as a single instance

— Interview & sources

Facade vs Adapter?

Facade invents a simpler interface over many classes; Adapter makes one existing interface fit an expected one.

Facade vs Mediator?

Facade is one-way (clients to subsystem); Mediator is two-way coordination among peers that know it.

Does a facade hide the subsystem?

It can, but needn't — transparent facades still allow direct access for advanced cases.

When does it smell?

When it accrues business logic and becomes a god object instead of a thin delegating front.

REMEMBER

One coarse, simple interface over a complex subsystem — it delegates and composes, adding no behavior of its own.

SOURCES

GoF — "Design Patterns" (1994): Facade (structural)

Freeman & Robson — "Head First Design Patterns" (2nd ed., 2020): Adapter & Facade